Introduction to Artificial Neural Networks with Keras

Hands-on Machine Learning with Python (Aurelien Geron)

Giacomo Fiumara

Introduction

Introduction

- Neural networks are a core family of machine learning models inspired by interconnected processing units.
- They are flexible enough for classification, regression, vision, speech, and recommendation tasks.

Learning goals

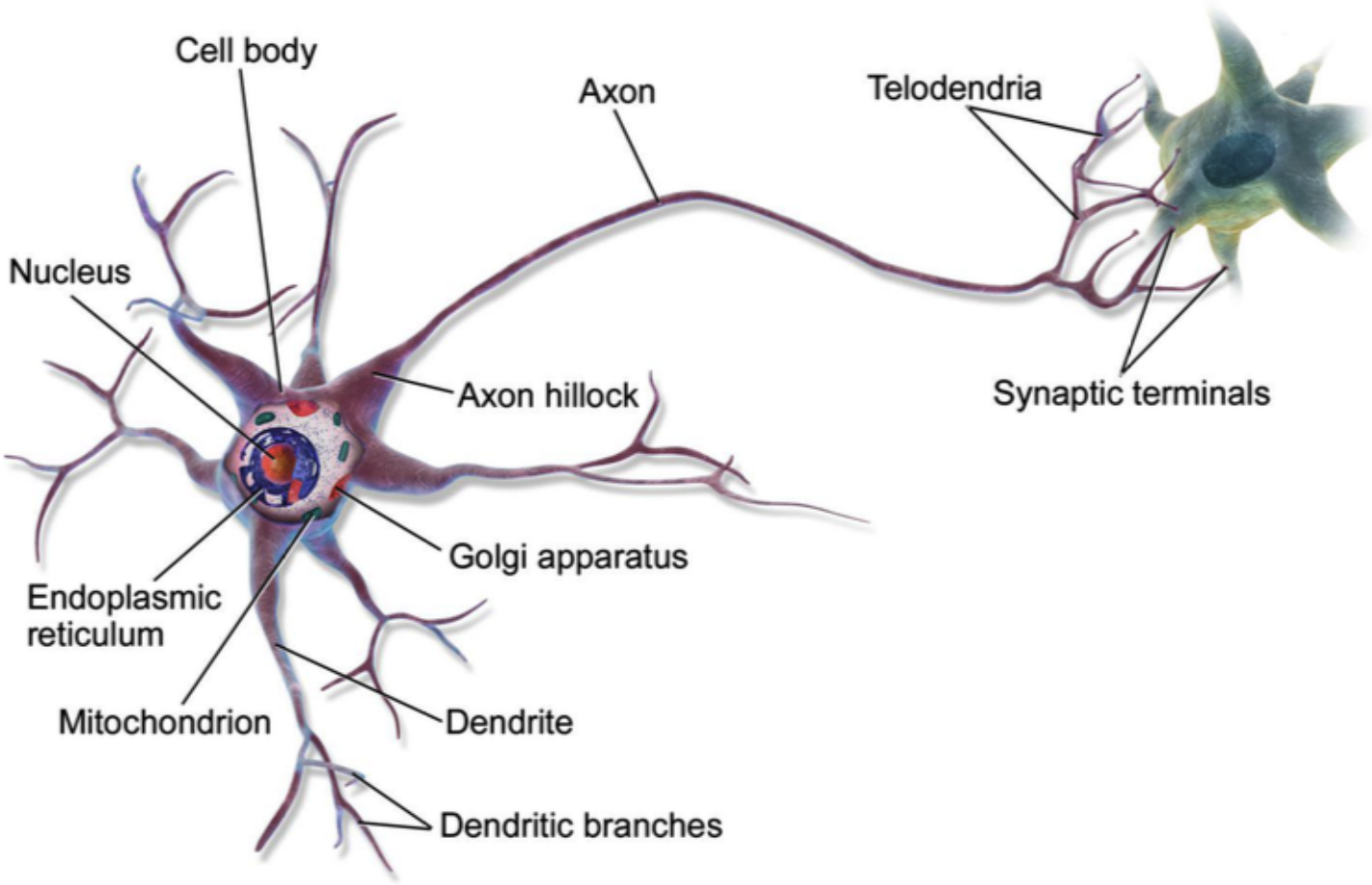
- Understand the intuition behind artificial neurons, perceptrons, multilayer perceptrons, and backpropagation.
- See why nonlinear activation functions are essential to model complex patterns.
- Learn the basic design choices for regression and classification MLPs with Keras.

From brains to models

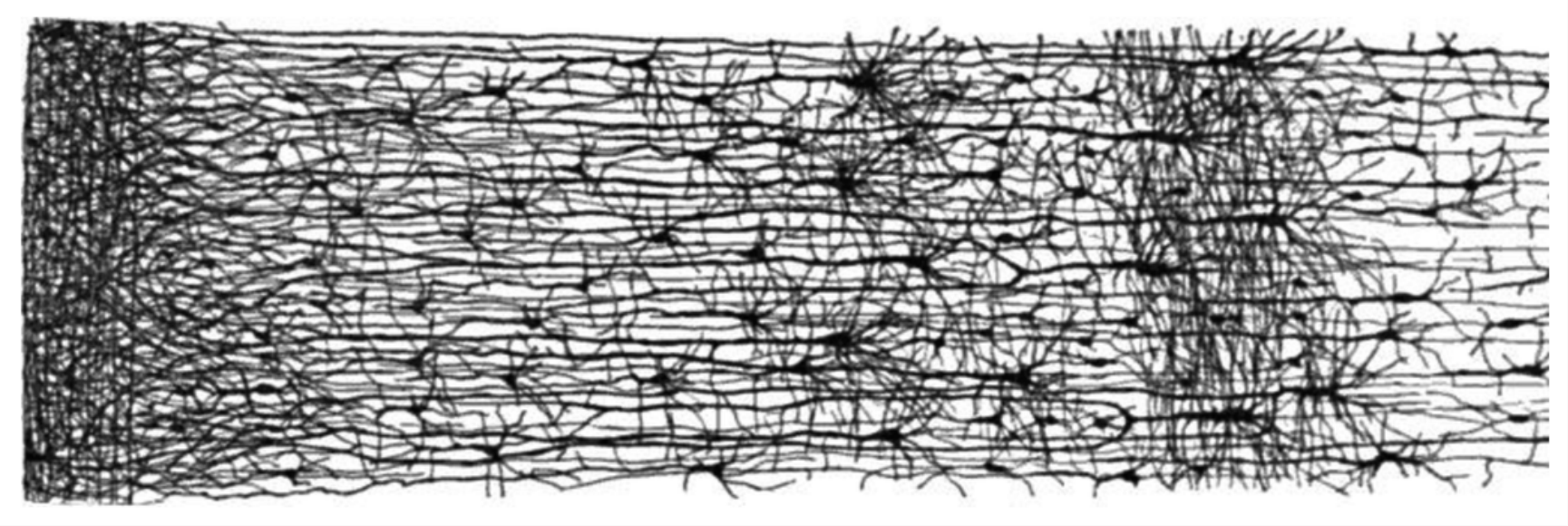
- The historical inspiration comes from biological neurons connected in large layered networks.
- Modern neural networks are not faithful brain simulations: they are mathematical models trained from data.
- The key idea is simple units plus many connections can produce rich collective behavior.

The neuron

Biological neuron



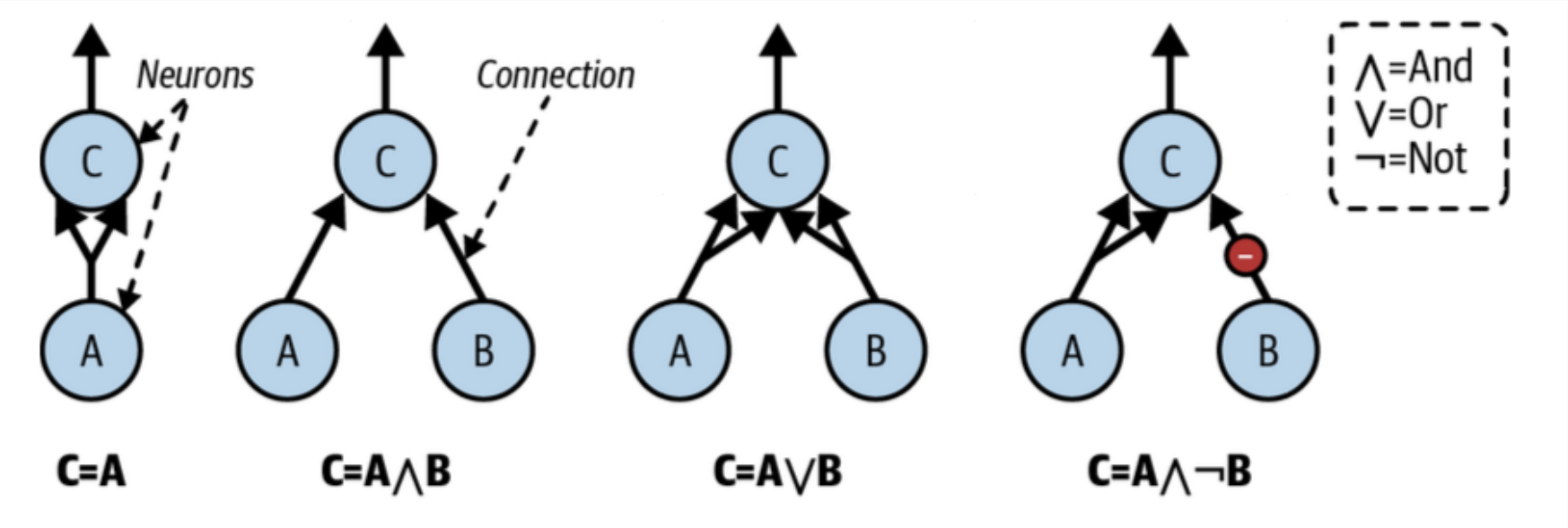
Multiple layers in biological networks



Logical view of early neurons

- The earliest artificial neuron models used binary inputs and binary outputs: on or off.
- By combining simple units, it is possible to implement logical operations such as identity, AND, OR, and NOT.
- This showed that networks of simple units can perform nontrivial computation.

Logical computations with neurons



In these examples, a neuron is activated if and only if two input connections are active

The perceptron

- A perceptron uses numeric inputs, one weight per input, and a bias term.
- It computes a weighted sum and then applies a threshold-like activation to decide the output.
- Geometrically, a perceptron defines a linear decision boundary in the input space.

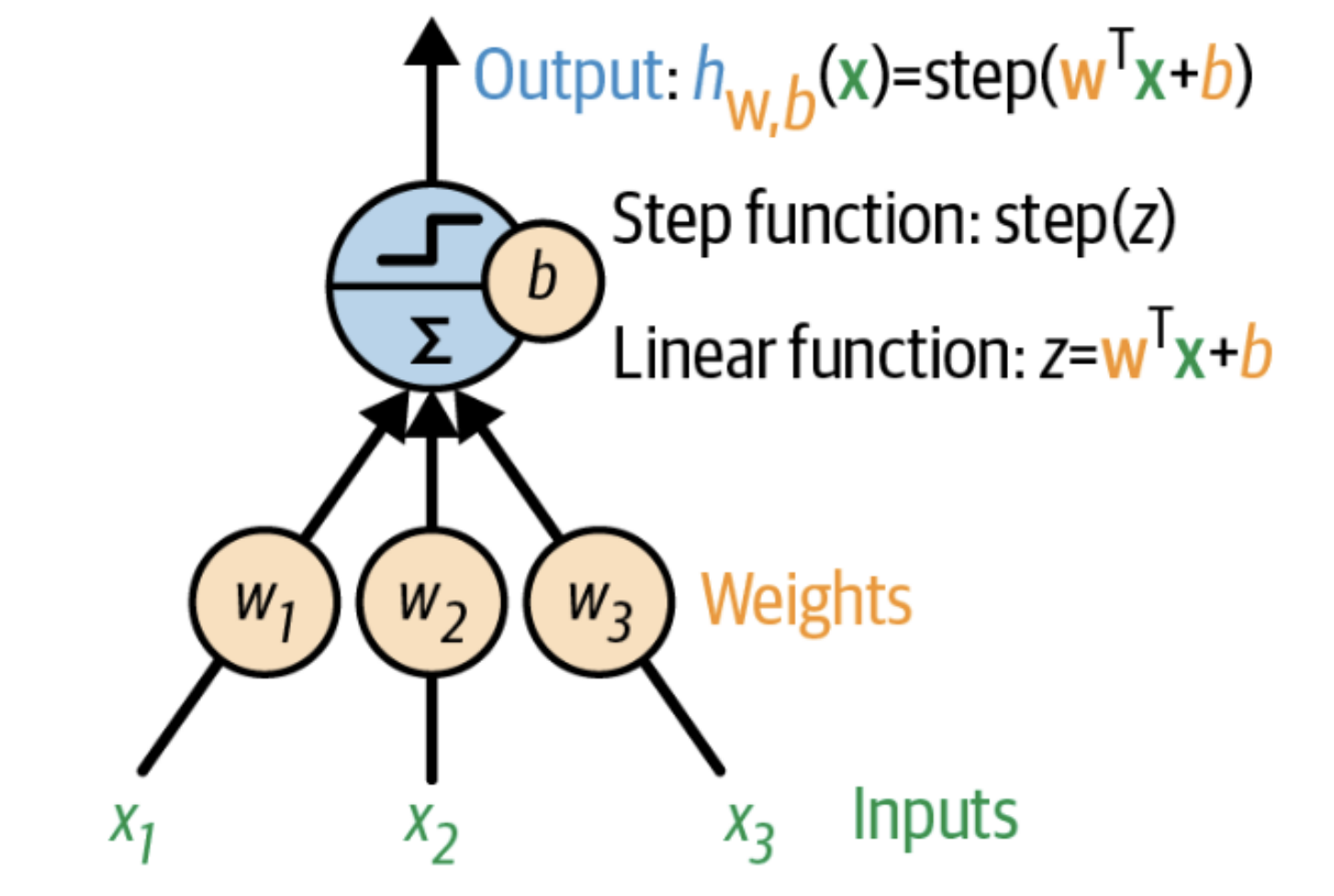
Weighted sum and threshold

- For one neuron, the pre-activation is

$$z = w_1 x_1 + \cdots + w_n x_n + b = \mathbf{w}^T \mathbf{x} + b$$

- A step function turns this score into a class decision, for example positive if $z \geq 0$.
- This makes the perceptron a close relative of linear classifiers such as logistic regression.
- b is called **bias**. It is a set of trainable parameters added to the weighted input of neurons in a neural network layer before the activation function is applied. It allows models to shift activation functions to the left or right, enabling the learning of non-linear patterns that do not pass through the origin.

Threshold logic unit



Other step functions

- Other step functions used in perceptrons are:

$$H(z) = \begin{cases} 0 & \text{if} \quad z < 0 \\ 1 & \text{if} \quad z \geq 0 \end{cases}$$

$$sgn(z) = \begin{cases} -1 & \text{if} \quad z < 0 \\ 0 & \text{if} \quad z = 0 \\ 1 & \text{if} \quad z > 0 \end{cases}$$

Applications of TLU

- A single TLU can be used for simple linear binary classification. It is a linear function of its inputs, and if the result exceed a threshold, it outputs the positive class. Otherwise the negative class.

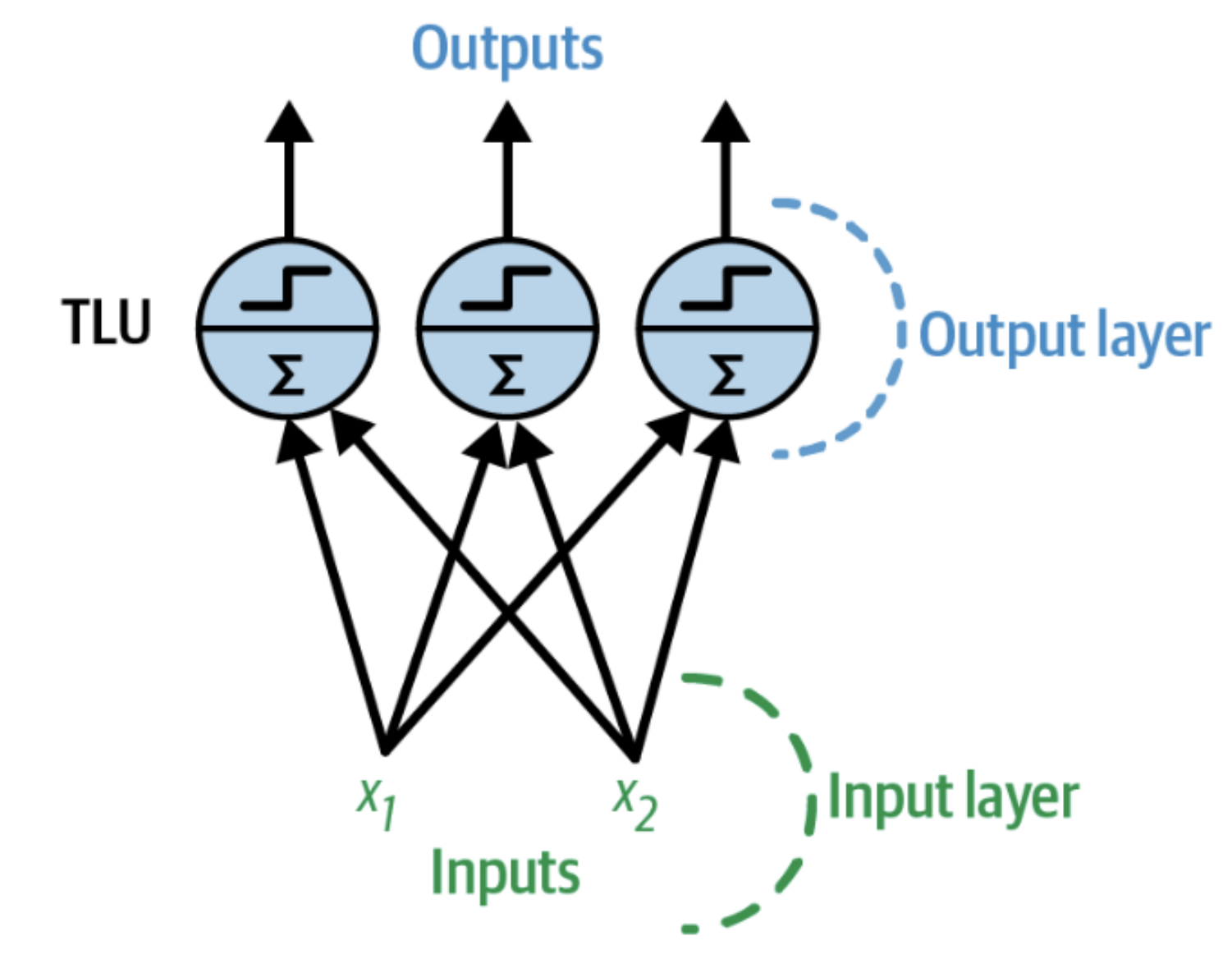
- For example, a single TLU can be used to classify iris flowers based on petal length and width. It is necessary to determine the right values of w_1, w_2 and b

The perceptron

A layer of perceptrons

- A perceptron is composed of one or more TLUs organized in a single layer, every TLU being connected to every input (**fully connected layer** or **dense layer**).
- With multiple output neurons, the model can produce several binary decisions at once.
- Matrix notation makes this compact and efficient for many instances processed together.

Perceptron architecture (example)



Forward computation in matrix form

- For a dense layer, the outputs are written as $\mathbf{XW} + \mathbf{b}$, followed by an activation function.
- $\mathbf{X}$ stores the mini-batch, $\mathbf{W}$ the weights, and $\mathbf{b}$ the bias vector.
- This notation is central because modern deep learning libraries compute whole batches this way.

How the perceptron learns /1

- The perceptron learning rule updates weights when a training example is misclassified.
- Connections that would support the correct answer are strengthened, while harmful ones are weakened.
- If the data are linearly separable, the perceptron is guaranteed to find a separating solution.

How the perceptron learns /2

- The perceptron is fed one training instance at a time, and for each instance it makes its predictions.
- For every output neuron that produces a wrong prediction, it reinforces the connection weights from the inputs that would have contributed to the correct prediction

How the perceptron learns /3

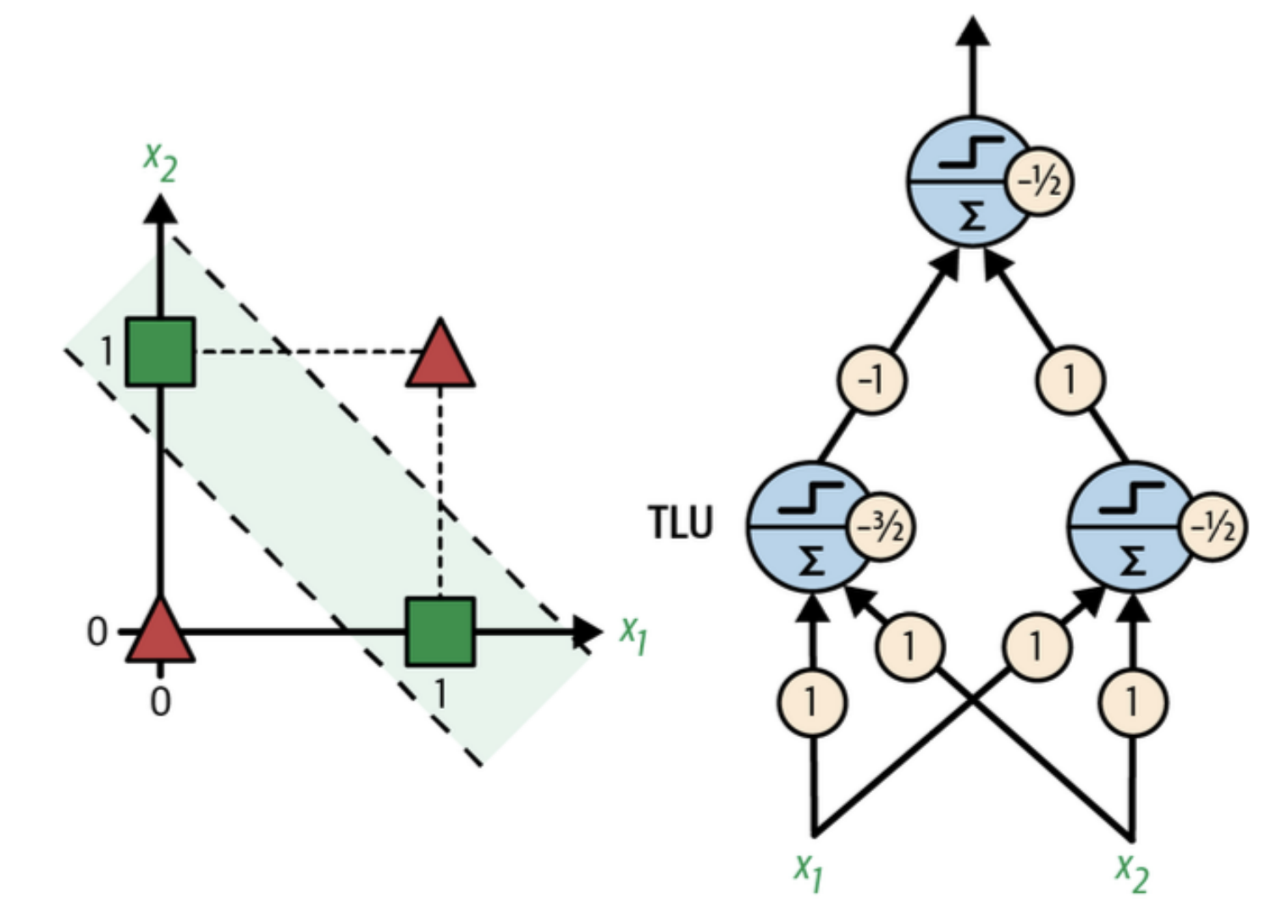
$$w_{i,j}^{\text{next step}} = w_{i,j} + \eta(y_j - \hat{y}_j)x_i$$

- $w_{i,j}$ is the connection weight between the i-th input and the j-th neuron
- x_i is the i-th input value of the current training instance
- $\hat{y}_j$ is the output of the j-th output neuron for the current training instance
- y_j is the target output of the j-th output neuron for the current training instance
- η is the learning rate

Limits of perceptrons

- A perceptron can only learn linear boundaries, so some simple tasks remain impossible.
- The classic example is XOR, where no single straight line can separate the two classes.
- This limitation motivated the move from shallow linear models to multilayer networks.

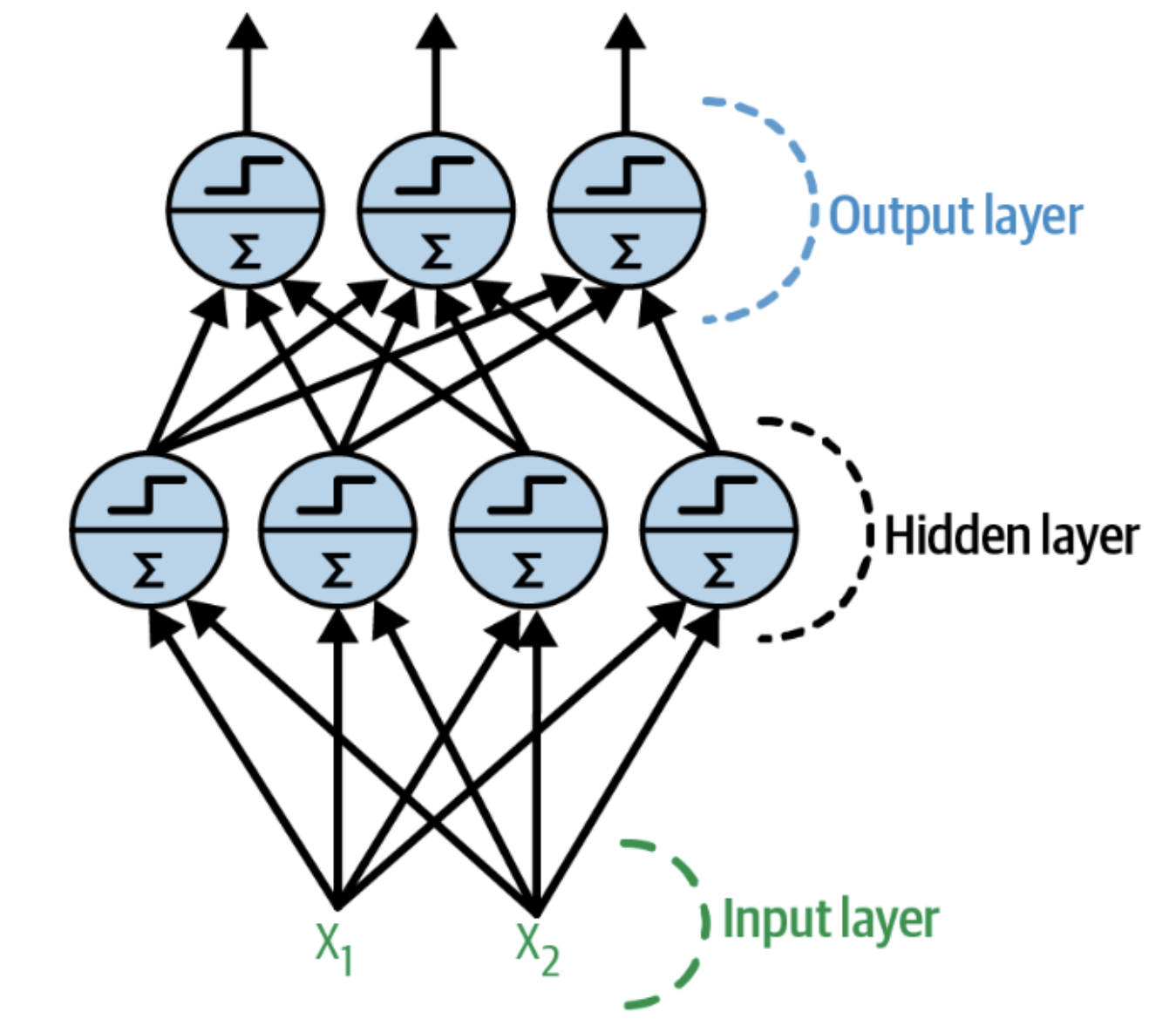
XOR and MLP solution



From perceptron to MLP

- A **M**ulti**L**ayer **P**erceptron (MLP) has an input layer, one or more hidden layers, and an output layer.
- Hidden layers allow the network to build intermediate representations of the data.
- With enough nonlinear hidden units, an MLP can represent far more complex functions than a perceptron.

Multilayer perceptron



Feedforward

- Information flows from the inputs to the outputs, layer by layer, without cycles.
- Each layer transforms the previous representation into a new one that is often more useful.
- Early layers tend to detect simple patterns, while higher layers combine them into richer concepts.

Why deep can help

- In principle, one hidden layer can approximate very complex functions if it is large enough.
- In practice, deeper networks often represent complex patterns more efficiently than very wide shallow ones.
- The hierarchy of layers matches many real problems where simple features combine into complex structures.

Backpropagation

Backpropagation

- Backpropagation computes how much each weight contributed to the error, then updates weights to reduce it.
- It combines a forward pass, a loss computation, a backward pass, and an optimization step.
- This is the standard training mechanism behind most neural networks.

Backpropagation: the basic loop

- First, a mini-batch goes forward through the network and produces predictions.
- Then the loss compares predictions with targets, giving a numerical measure of error.
- Finally, gradients flow backward and the optimizer adjusts the parameters.

Gradient descent: the core question

- Training an MLP means finding weights and biases that make the network's predictions match the targets.
- We therefore need a method to minimize a loss function over a huge number of parameters.
- Gradient descent is the practical answer because it tells us how to improve all parameters together.

What training really means

- An MLP is a parametrized function $f_\theta(x)$, where θ contains all weights and biases.
- A loss function $L(\theta)$ measures how wrong the network is on the training data.
- Training is an optimization problem: find parameter values that make $L(\theta)$ as small as possible.

Why this is difficult

- Even a small MLP may contain thousands of parameters, while real models may contain millions or more.
- The loss is a nonlinear function of these parameters because layers and activations are composed together.
- So we cannot solve training by simple inspection or by trying all possibilities.

Why not a closed-form solution?

- In linear regression, the objective has a special structure and can often be solved analytically.
- In an MLP, the composition of many nonlinear layers destroys this simplicity.
- As a result, there is no practical closed-form formula for the optimal weights of a general MLP.

Why not brute force search?

- Searching directly in parameter space is impossible because the number of candidate configurations is enormous.
- If each parameter could take only a few values, the number of combinations would still explode exponentially.
- We need a local rule that says how to move from the current parameters to better ones.

The role of the gradient

- The gradient of the loss collects all partial derivatives with respect to the model parameters.
- It tells us how sensitive the loss is to each weight and bias around the current point.
- The negative gradient gives the direction of steepest local decrease of the loss.

The update rule

- Gradient descent updates the parameters using

$$\theta^{\text{new}} = \theta^{\text{old}} - \eta \nabla_\theta L(\theta^{\text{old}})$$

- The learning rate η controls the step size: too small is slow, too large may become unstable.

- Repeating this update gradually moves the model toward lower-loss regions.

The basic idea

- Imagine the loss function as a high-dimensional surface with hills, valleys, and plateaus.
- The current parameter vector is one point on this landscape, and the gradient is the local slope.
- Gradient descent uses that slope information to choose a downhill direction instead of moving blindly.

Why the gradient is especially valuable

- A single gradient vector summarizes how every parameter should change to reduce the loss.
- This is much more informative than random search, which has no local directional guidance.
- In high dimensions, using local slope information is what makes optimization feasible.

Where backpropagation enters

- Gradient descent needs the gradient, but an MLP has many layers and many parameters.
- Backpropagation computes these derivatives efficiently by applying the chain rule from the output back to the input.
- In short: backpropagation computes the gradients, gradient descent uses them to update parameters.

Forward pass and backward pass

- In the forward pass, the network computes predictions and then the loss on a mini-batch.
- In the backward pass, backpropagation computes how each parameter contributed to the final error.
- Once these gradients are available, the optimizer performs one update step.

Why backpropagation is essential

- Without backpropagation, we could estimate derivatives numerically, but that would be far too expensive.
- Numerical differentiation would require separate perturbations for huge numbers of parameters.
- Backpropagation reuses intermediate computations, making gradient-based training practical.

Why not use step activations?

- Gradient descent relies on derivatives, so activations must provide useful gradient information.
- A hard step function is flat almost everywhere, which means its gradient is zero or undefined.
- This is why trainable MLPs use differentiable or almost differentiable activations such as sigmoid, tanh, or ReLU.

Why nonlinearity matters

- If every layer were purely linear, the whole network would still collapse to one linear transformation.
- Nonlinear activations let the MLP represent complex decision boundaries and rich functions.
- Gradient descent then optimizes these nonlinear models by following local derivative information.

Do we really find the global optimum?

- Not necessarily: the loss of an MLP is generally non-convex, so there may be many local minima and saddle points.
- In practice, however, a very good solution is often enough, even if it is not the absolute global minimum.
- This is one reason gradient-based methods remain effective despite the complexity of neural network losses.

Why gradient descent still works well in practice

- Many bad optimization obstacles are less problematic in large neural networks than one might first expect.
- Stochasticity from mini-batches often helps the optimizer move away from sharp traps and saddle regions.
- Good initialization, suitable activations, and adaptive optimizers further improve training behavior.

Batch gradient descent vs SGD

- Full-batch gradient descent uses the whole dataset for each update, which can be expensive.
- Stochastic gradient descent uses one instance at a time, while mini-batch methods use small subsets.
- Mini-batch training is the standard compromise between computational efficiency and stable learning.

Why mini-batches are standard

- They exploit vectorized computation efficiently on GPUs and modern hardware.
- They provide gradient estimates that are cheaper than full-batch gradients and less noisy than single-instance updates.
- This makes mini-batch gradient descent the standard training regime for modern neural networks.

What if we used random updates instead?

- Randomly changing weights would sometimes improve the loss, but most updates would be wasted.
- The larger the number of parameters, the less plausible it becomes to improve them by chance.
- The gradient gives principled local information, which is why it is dramatically more efficient.

Main advantages of gradient descent for MLPs

- It scales to very high-dimensional parameter spaces.
- It exploits derivative information instead of searching blindly.
- Together with backpropagation, it turns multilayer neural network training into a feasible computation.

Gradient descent: summary

- The MLP defines a complex loss surface over its weights and biases.
- Backpropagation computes the gradient of that surface efficiently.
- Gradient descent uses this gradient to move the parameters toward lower error.
- We use gradient descent not because it is mathematically perfect, but because it is the practical method that works.
- Without gradient-based optimization, training general multilayer perceptrons would be computationally unrealistic.

Activation functions

Why differentiability matters

- Gradient-based optimization needs derivatives, so flat step functions are problematic for training.
- Replacing hard thresholds with smooth or piecewise differentiable activations makes learning feasible.
- This change was crucial for turning MLPs into practical trainable models.

Activation functions: sigmoid

- The sigmoid maps inputs to $(0, 1)$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

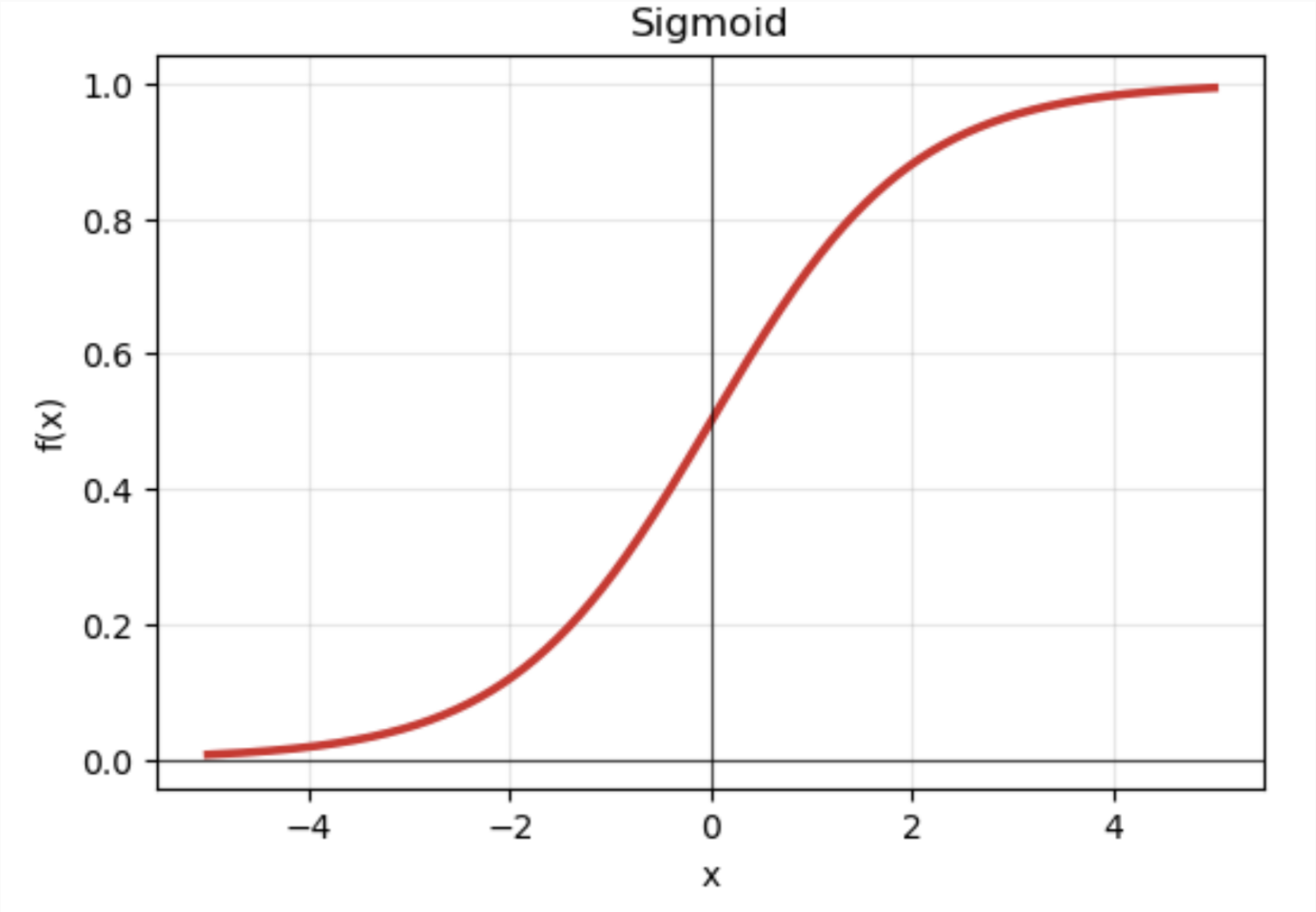
- The tanh maps inputs to $(-1, 1)$

$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

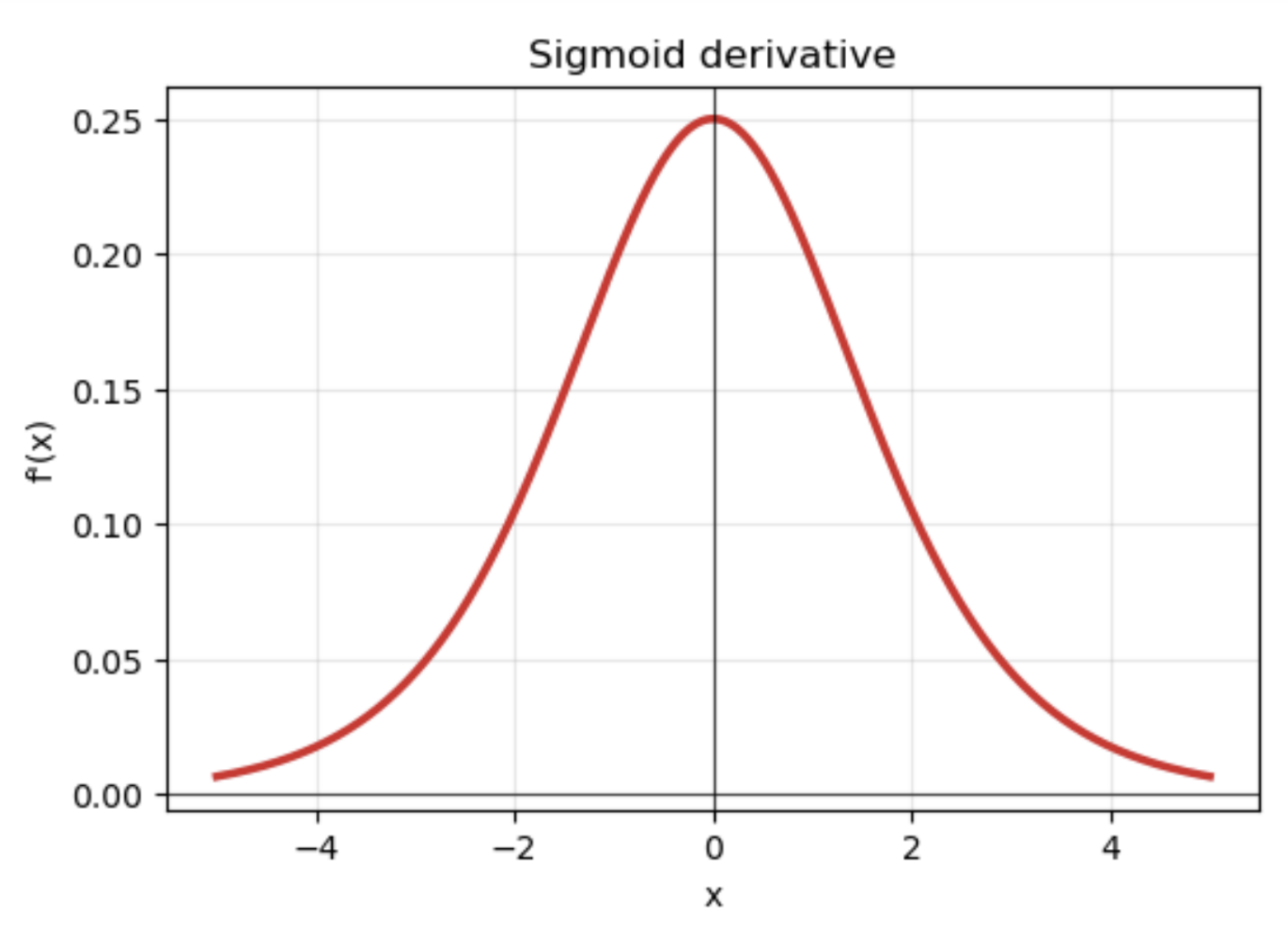
- The ReLU computes $\max(0, z)$

$$ReLU(z) = max(0, z)$$

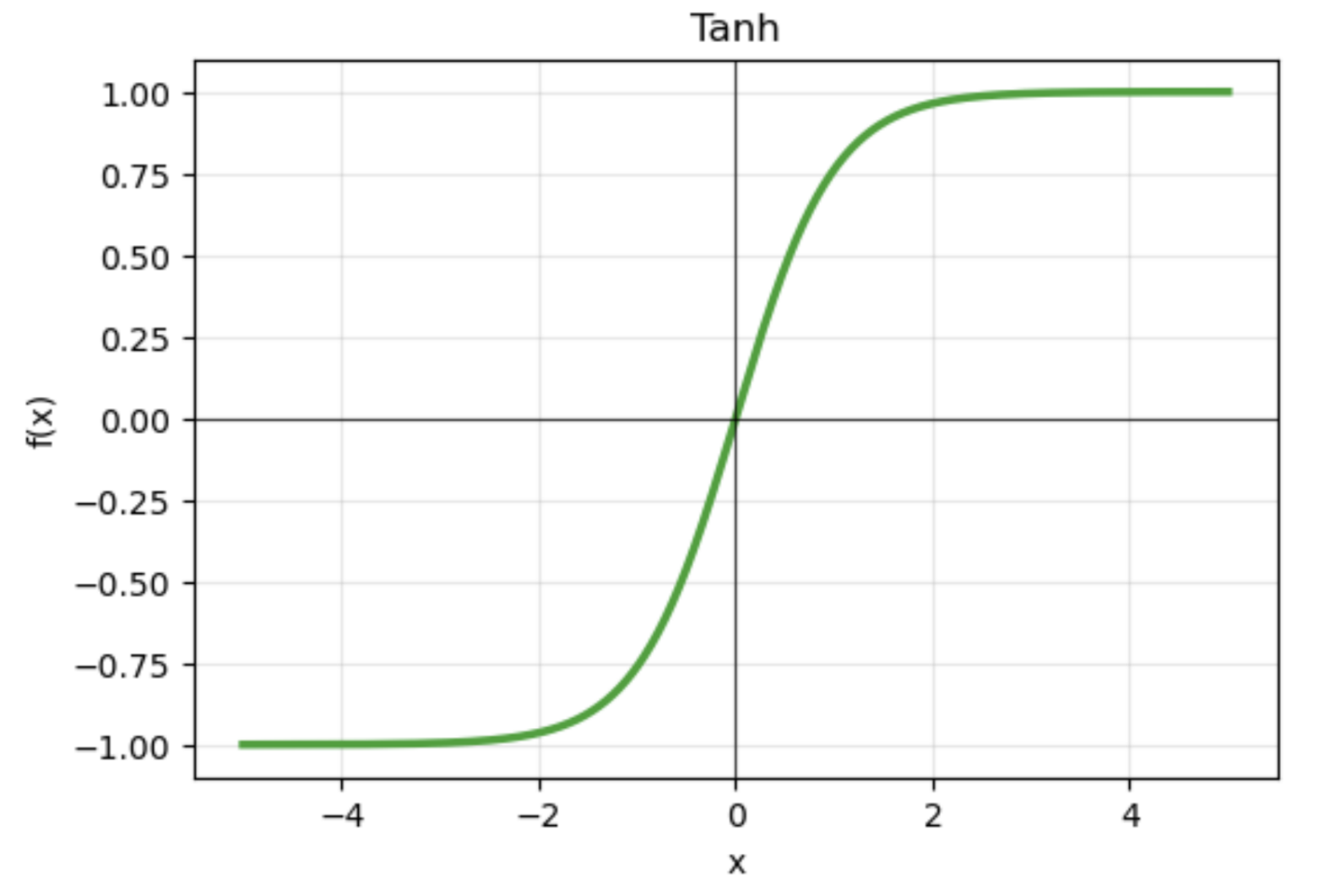
Activation functions: sigmoid



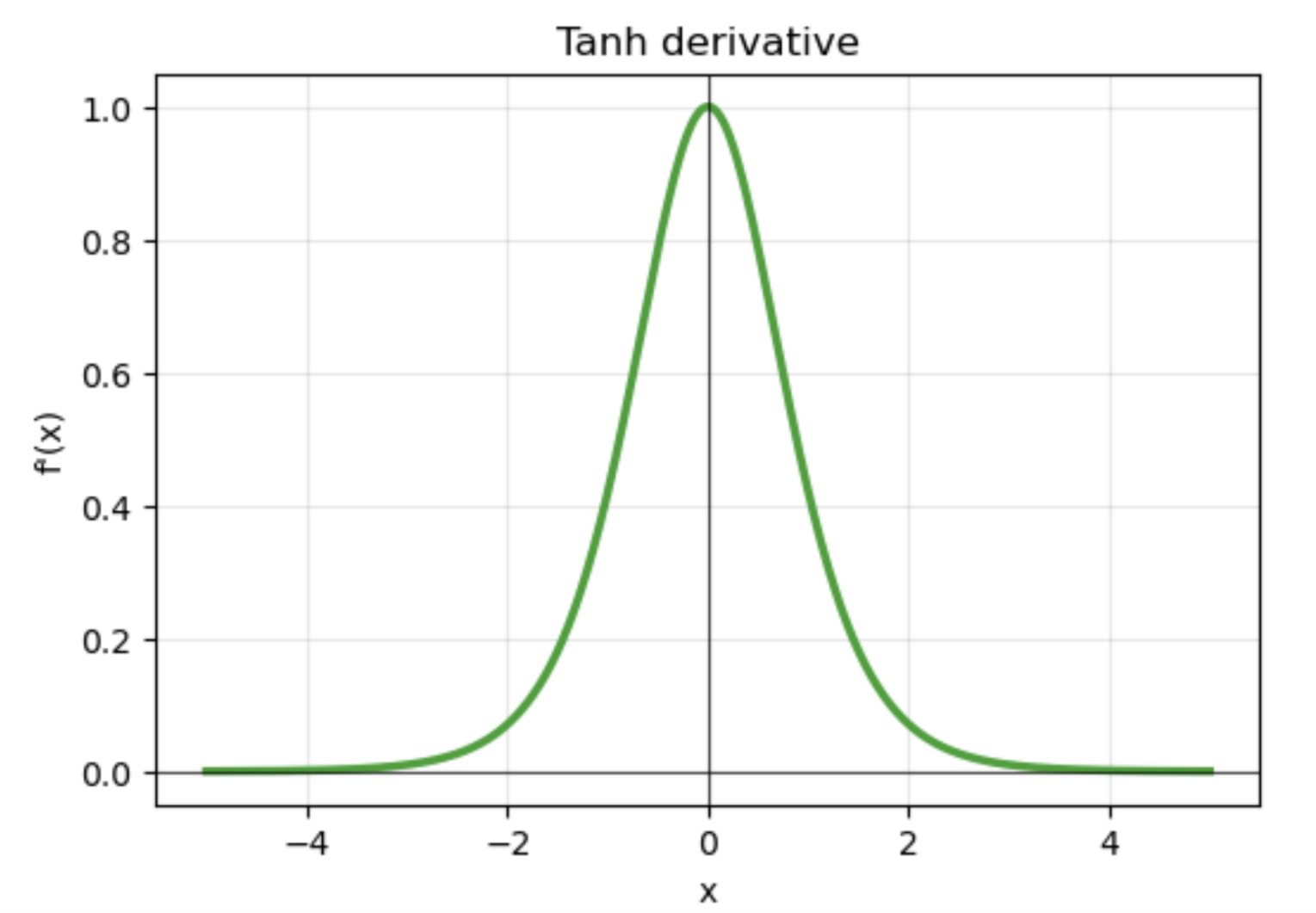
Activation functions: sigmoid



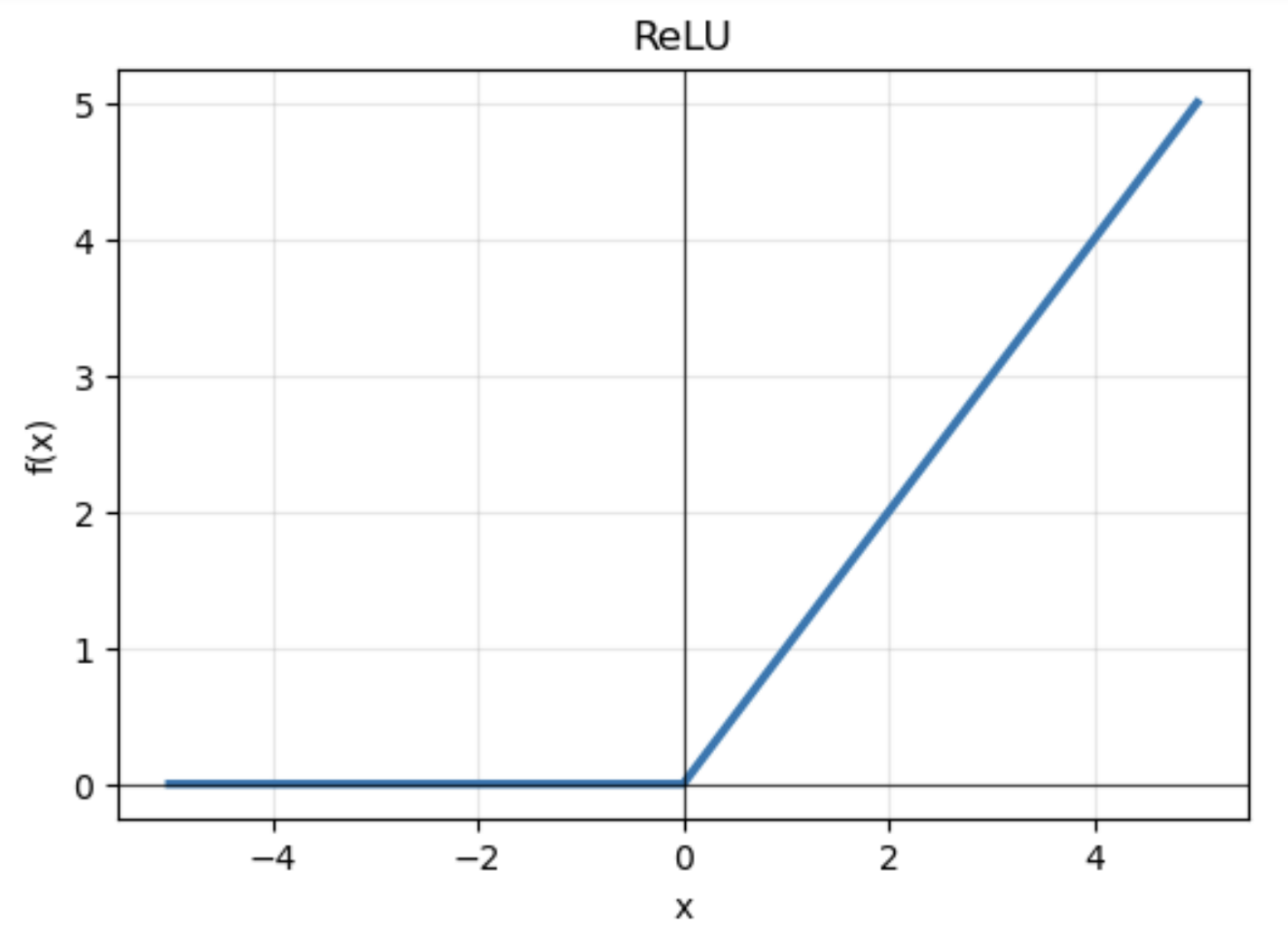
Activation functions: tanh



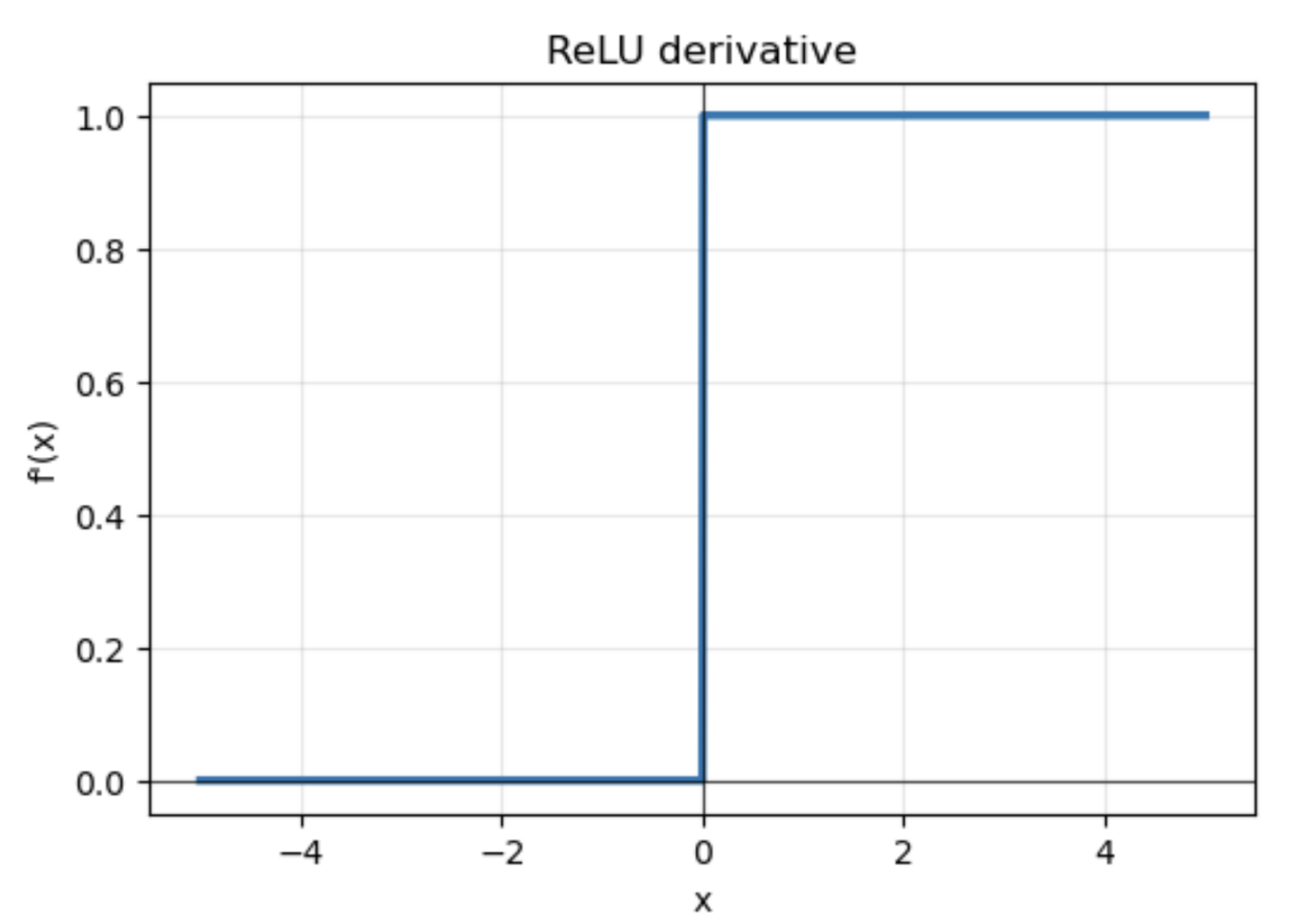
Activation functions: tanh



Activation functions: ReLU



Activation functions: tanh



Why ReLU became the default

- ReLU is computationally cheap and often trains faster than sigmoid-like activations in hidden layers.
- Its positive side does not saturate, which helps optimization in many practical models.
- For introductory MLPs, a good default is ReLU in all hidden layers.

Motivation

- Deep models are trained by backpropagation, so optimization depends critically on how gradients flow through many layers.
- Saturating nonlinearities such as sigmoid or tanh can shrink gradients and slow down learning in deep networks.
- ReLU became dominant because its positive branch is linear and non-saturating, which changes this gradient behavior.
- The key question is why this simple property makes optimization easier in practice.

What saturation means

- An activation saturates when large input changes produce only tiny output changes.
- In saturated regions, the derivative is close to zero, so the backward signal is strongly attenuated.
- If this happens repeatedly across many layers, early layers receive very small updates.
- This is one of the classic mechanisms behind the vanishing gradient problem.

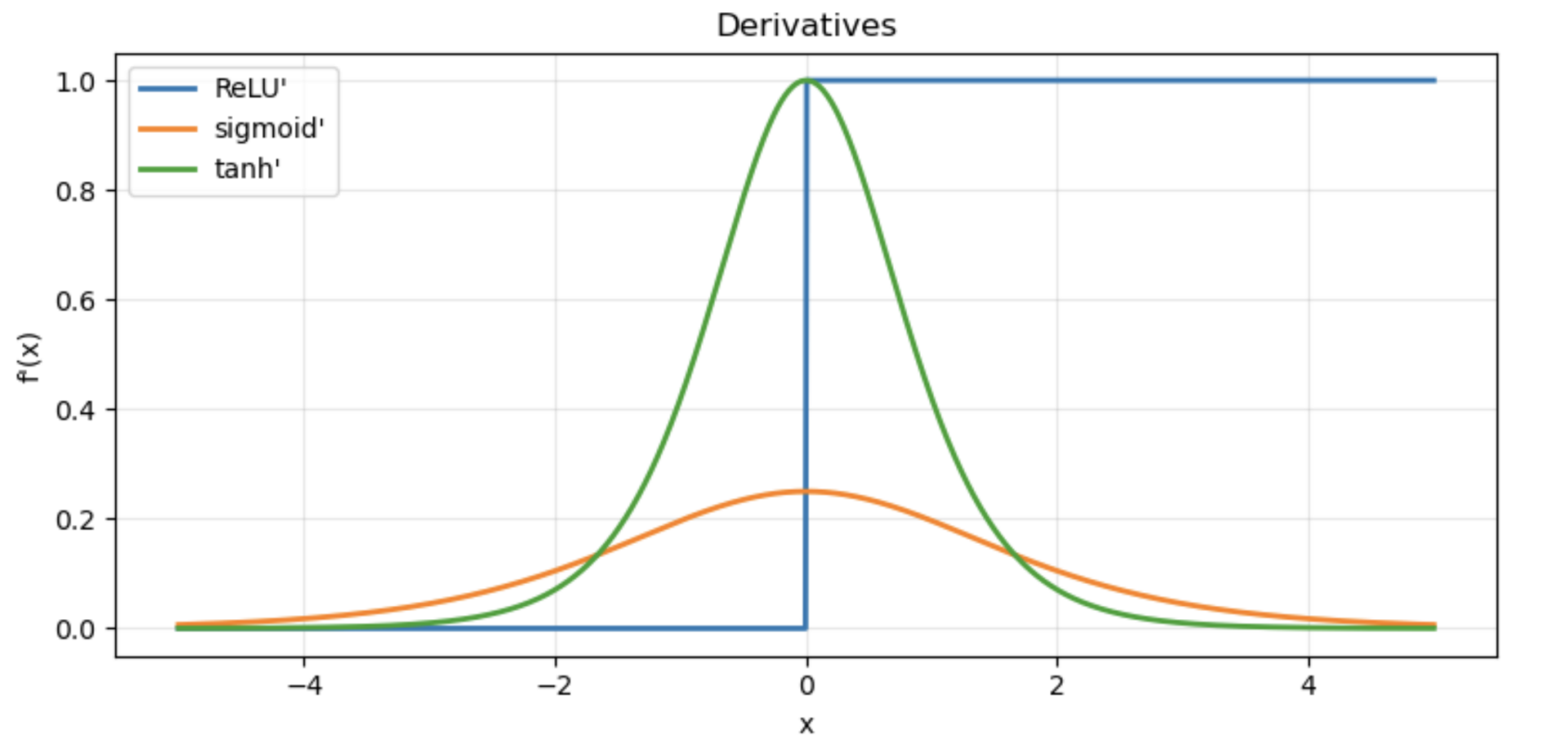
Sigmoid and tanh behavior

- For sigmoid, the derivative is maximized near zero and becomes tiny for large positive or negative inputs.
- Tanh is zero-centered, but it also saturates for large absolute inputs.
- As activations drift into these regions, gradients become progressively smaller layer after layer.
- This makes deep optimization sensitive to initialization and often much slower.

Derivative comparison

- On the positive side, ReLU has derivative 1, so the local gradient is preserved instead of compressed.
- Sigmoid and tanh have derivatives strictly below 1, and often much closer to 0 in practice.
- During backpropagation, these local derivatives are multiplied across layers.
- A chain of factors near 1 is far easier to optimize than a chain of factors that keep shrinking.

Activation functions: derivatives



Backpropagation intuition

- In one layer, the gradient is scaled by the derivative of the activation evaluated at that layer's pre-activation.
- If many derivatives are small, the product becomes tiny before reaching early layers.
- If active ReLU units contribute derivatives of 1, the activation itself does not cause further decay on those paths.
- Optimization is therefore easier because useful gradient paths remain open.

Simple chain example

- Suppose the backward signal crosses 10 layers.
- If each activation contributes a factor of 0.2, the total scale is $0.2^{10} \approx 10^{-7}$.
- If active ReLU units contribute a factor of 1, the activation does not shrink the signal along those paths.
- This does not solve every optimization issue, but it removes a major source of gradient attenuation.

Why the positive branch matters

- The positive branch does not flatten as input grows, unlike sigmoid or tanh.
- Large positive pre-activations therefore remain trainable instead of entering a nearly frozen regime.
- Units that are ON can keep transmitting both activations forward and gradients backward efficiently.
- This is the central optimization advantage of ReLU.

Sparse active paths

- ReLU also sets negative inputs to zero, creating sparse activations.
- Optimization can still work well as long as gradient can propagate along some active paths.
- Credit and blame are then assigned to units that are ON, instead of being diffusely spread across all units.
- This selective routing may make optimization simpler and more stable.

Interaction with initialization

- Because active ReLU units preserve slope on the positive side, deep networks become less fragile to poor gradient scaling.
- This helped make very deep architectures trainable when combined with suitable initialization schemes.
- Later work on He initialization explicitly matched initialization to rectifier networks.
- In practice, ReLU reduced the burden imposed by sigmoidal saturation during early optimization.

Computational simplicity

- ReLU is also cheap to evaluate: it is just a threshold at zero.
- Its derivative is simple as well, which reduces computational overhead in training.
- Although this is not the main reason for better optimization, it makes large-scale training more efficient.
- Faster iterations often amplify the practical advantage of better gradient flow.

What ReLU does not fix

- ReLU does not eliminate all optimization problems in deep learning.
- Units can become permanently inactive, leading to the dying ReLU problem.
- The function is unbounded on the positive side, so activation magnitude can still become large without normalization.
- This is why ReLU is often paired with initialization, normalization, and careful learning-rate choices.

Common variants

- Leaky ReLU and PReLU keep a small nonzero slope for negative inputs to reduce dead units.
- ELU and related activations try to preserve some optimization benefits while improving activation statistics.
- These variants modify the negative branch more than the positive branch.
- The original ReLU advantage still comes primarily from the non-saturating positive side.

Regression with MLPs

- For regression, the network predicts one or more continuous numerical outputs.
- A common choice is one output neuron per predicted quantity and mean squared error as the loss.
- Input standardization is especially important because training relies on gradient-based optimization.

Typical regression design choices

- Hidden layers often use ReLU, while the output layer may use no activation for unconstrained targets.
- If targets must be positive, bounded, or robust to outliers, output activation and loss may change.
- The architecture depends on the problem, but small MLPs are often enough for tabular regression.

Scikit-Learn example: MLPRegressor

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

housing = fetch_california_housing()
X_train, X_test, y_train, y_test = train_test_split(
housing.data, housing.target, random_state=42)

model = make_pipeline(
StandardScaler(),
MLPRegressor(hidden_layer_sizes=(50, 50, 50),
random_state=42)
)
model.fit(X_train, y_train)
```

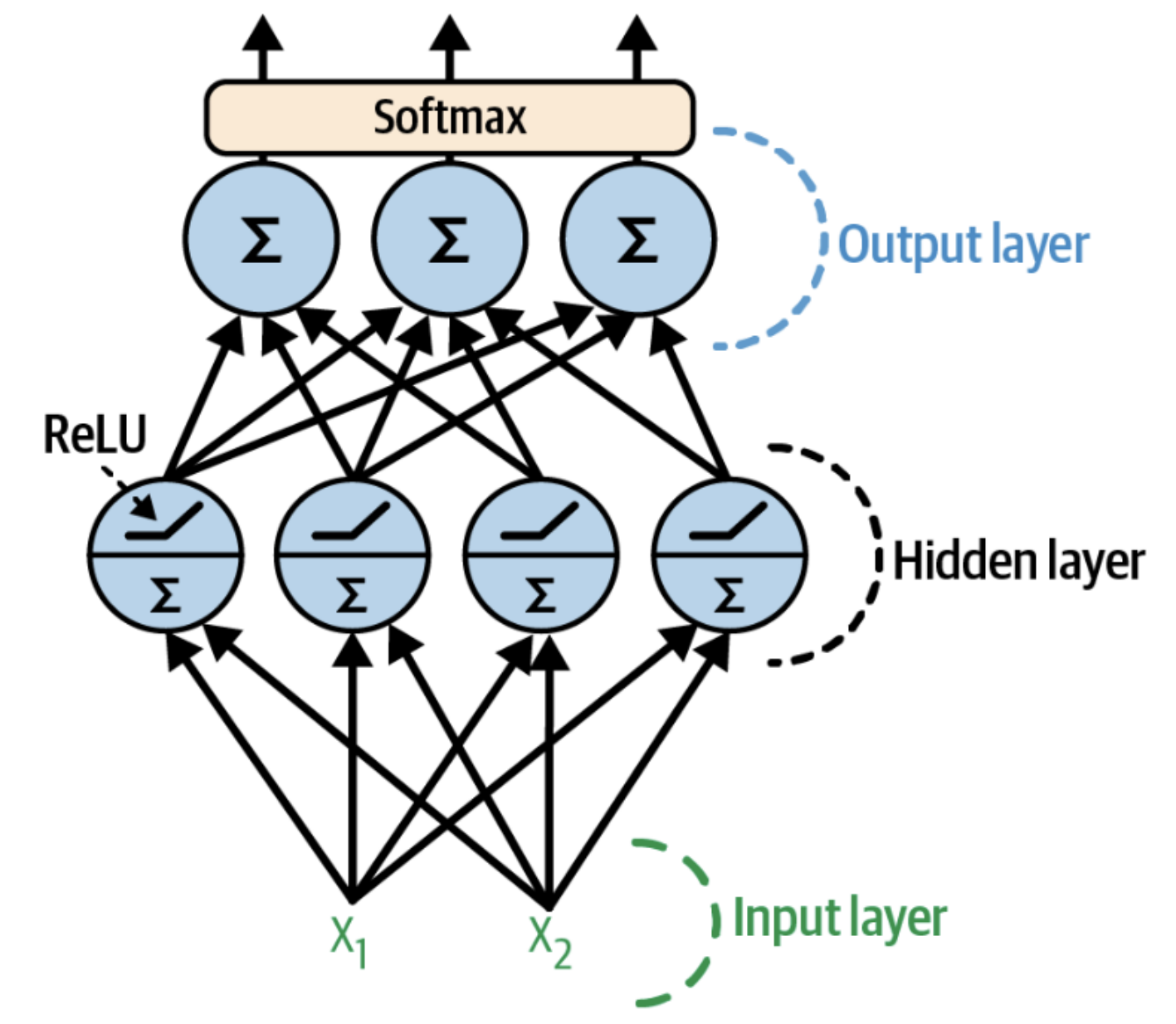
Classification with MLPs

- For binary classification, one output neuron with sigmoid gives the probability of the positive class.
- For multiclass problems, we usually use one output neuron per class and a softmax output layer.
- Cross-entropy is the standard loss because the model predicts class probabilities.

Binary, multilabel, multiclass

- Binary: one class decision, usually one sigmoid output neuron.
- Multilabel: one sigmoid output per label, since labels are not mutually exclusive.
- Multiclass: one softmax output vector, since exactly one class is assumed to be correct.

MLP for classification



Output layer choices at a glance

- Regression: often linear output, one neuron per predicted variable.
- Binary classification: one sigmoid output, optimized with cross-entropy.
- Multiclass classification: one softmax output layer, one neuron per class.

Scikit-Learn example: MLPClassifier

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
iris.data, iris.target, random_state=42)

clf = make_pipeline(
StandardScaler(),
MLPClassifier(hidden_layer_sizes=(10,), random_state=42)
)
clf.fit(X_train, y_train)
```

Softmax and cross-entropy

Where they appear

- Softmax and cross-entropy are standard tools for multiclass classification, where each input belongs to exactly one class.
- scikit-learn's `MLPClassifier` uses cross-entropy loss and supports probability estimates through `predict_proba()`.
- TensorFlow provides dedicated APIs such as `softmax_cross_entropy_with_logits` for mutually exclusive classes.

From scores to probabilities

- A neural network usually outputs raw scores $z_1, \ldots, z_K$, often called *logits*.
- Logits are not probabilities: they can be negative, larger than 1, and do not sum to 1.
- We need a mapping that turns arbitrary real values into nonnegative values summing to 1 across classes.

Softmax definition

$$\hat{y}_i = \mathrm{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

- Each output $\hat{y}_i$ is in $(0, 1)$, and $\sum_i \hat{y}_i = 1$.
- Large logits get amplified exponentially, so the highest score tends to dominate the distribution.
- Softmax is therefore a smooth alternative to taking only the $\arg\max$.

Softmax intuition

- If one logit is much larger than the others, its probability becomes close to 1.
- If several logits are similar, softmax distributes probability mass more evenly across them.
- The transformation preserves ranking: the largest logit remains the most probable class.

An example

Suppose the logits are

$$\mathbf{z} = [2.0,\ 1.0,\ 0.1]$$

Then

$$\mathrm{softmax}(\mathbf{z}) \approx [0.659,\ 0.242,\ 0.099]$$

- The model prefers class 1, but it still assigns some probability to the other two classes.
- This is useful during training because gradients depend on the full distribution, not only on the winning class.

True labels in multiclass tasks

- In multiclass classification, the target is often represented as a one-hot vector.
- For class c, the true vector is $\mathbf{y} = [0, \ldots, 1, \ldots, 0]$, with a single 1 at position c.
- The loss should reward high probability on the correct class and penalize probability assigned elsewhere.

Cross-entropy definition

For one training example, categorical cross-entropy is

$$L(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{i=1}^{K} y_i \log(\hat{y}_i)$$

- This is the standard loss for multiclass classification with softmax outputs.
- When targets are one-hot encoded, only the correct class contributes to the sum.
- The loss is small when the model assigns high probability to the true class.

Cross-entropy with one-hot labels

If the correct class is c, then

$$L = -\log(\hat{y}_c)$$

- If $\hat{y}_c = 0.9$, the loss is low.
- If $\hat{y}_c = 0.1$, the loss is much larger.
- Cross-entropy strongly penalizes being confidently wrong.

Numerical interpretation

Predicted $\hat{y}_c$	Loss $-\log(\hat{y}_c)$	Interpretation
0.90	0.105	correct and confident
0.50	0.693	uncertain
0.10	2.303	mostly wrong
0.01	4.605	confidently wrong

- The logarithm makes the penalty grow rapidly when the true class gets very small probability.

Why softmax and cross-entropy fit together

- Softmax provides a proper probability distribution over mutually exclusive classes.
- Cross-entropy compares that predicted distribution with the target distribution.
- Their combination yields a particularly simple gradient, which is crucial for backpropagation.

Key gradient result

Let $\mathbf{z}$ be the logits and $\hat{\mathbf{y}} = \mathrm{softmax}(\mathbf{z})$. Then

$$\frac{\partial L}{\partial z_i} = \hat{y}_i - y_i$$

- The gradient is simply prediction minus target.
- This elegant result makes optimization stable and efficient in neural networks.
- It also explains why probabilities of incorrect classes are pushed down while the correct class is pushed up.

Interpretation of the gradient

- For the correct class, $y_i = 1$, so the gradient is $\hat{y}_i - 1$: it is negative unless the model is already perfect.
- For the wrong classes, $y_i = 0$, so the gradient is just $\hat{y}_i$: larger wrong probabilities get larger penalties.
- Gradient descent therefore redistributes probability mass toward the correct class.

Numerical stability

- Computing e^{z_i} directly may overflow when logits are large.
- A common trick is to subtract the maximum logit before softmax: this does not change the probabilities.
- Modern libraries usually combine softmax and cross-entropy internally for better numerical stability.

Stable softmax formula

$$\hat{y}_i = \frac{e^{z_i - z_{\max}}}{\sum_{j=1}^{K} e^{z_j - z_{\max}}} \qquad z_{\max} = \max_j z_j$$

- Subtracting the same constant from all logits leaves softmax unchanged.
- This keeps exponentials in a safer numerical range.

NumPy implementation

```
import numpy as np

def softmax(z):
        z = z - np.max(z, axis=-1, keepdims=True)
        exp_z = np.exp(z)
        return exp_z / np.sum(exp_z, axis=-1, keepdims=True)

z = np.array([[2.0, 1.0, 0.1]])
p = softmax(z)
print(p)
```

Cross-entropy in NumPy

```
def cross_entropy(y_true, y_pred, eps=1e-12):
        y_pred = np.clip(y_pred, eps, 1.0)
        return -np.sum(y_true * np.log(y_pred), axis=-1)

y_true = np.array([[1, 0, 0]])
loss = cross_entropy(y_true, p)
print(loss)
```

TensorFlow preferred usage

```
import tensorflow as tf

model = tf.keras.Sequential([
       tf.keras.layers.Dense(64, activation="relu"),
       tf.keras.layers.Dense(10) # logits, no softmax here
])

loss_fn = tf.keras.losses.SparseCategoricalCrossentropy(
       from_logits=True
)
```

- A common practice is to output logits directly and let the loss function handle the stable computation.

TensorFlow low-level API

```
logits = tf.constant([[2.0, 1.0, 0.1]])
labels = tf.constant([[1.0, 0.0, 0.0]])

loss = tf.nn.softmax_cross_entropy_with_logits(
       labels=labels,
       logits=logits
)
```

scikit-learn connection

```
from sklearn.neural_network import MLPClassifier

clf = MLPClassifier(hidden_layer_sizes=(64,),
       random_state=42,
       max_iter=300)
clf.fit(X_train, y_train)
proba = clf.predict_proba(X_test)
```

- `MLPClassifier` uses cross-entropy loss and exposes class probabilities with `predict_proba()`.

Softmax vs sigmoid

- Softmax is used for *multiclass* problems with mutually exclusive classes.
- Sigmoid is used for binary classification or multilabel problems, where several labels may be active at once.
- In short: softmax couples the outputs; sigmoid treats each output independently.

Summary

- Softmax converts logits into a normalized class-probability distribution.
- Cross-entropy measures how far the predicted distribution is from the true label distribution.
- Together they form the standard objective for multiclass neural classification.
- The gradient $\hat{y} - y$ is the central reason why this combination is so effective in practice.

Keras

Why we move to Keras

- Scikit-Learn is excellent for simple experiments, but it offers limited neural network flexibility.
- Keras provides a high-level API to build, train, evaluate, save, and reuse neural networks.
- It is the standard entry point for practical deep learning with TensorFlow.

Keras in one view

- Keras lets us define models layer by layer, compile them with loss and optimizer, and fit them on data.
- It supports simple Sequential models as well as more flexible architectures.
- For beginners, it offers a clear path from intuition to executable neural network code.

Keras Sequential: minimal classifier

```
import tensorflow as tf

model = tf.keras.Sequential([
tf.keras.layers.Flatten(input_shape=[28, 28]),
tf.keras.layers.Dense(300, activation="relu"),
tf.keras.layers.Dense(100, activation="relu"),
tf.keras.layers.Dense(10, activation="softmax")
])

model.compile(loss="sparse_categorical_crossentropy",
optimizer="adam",
metrics=["accuracy"])
```

Training ingredients

- A neural network needs an architecture, a loss function, an optimizer, and data.
- During training, epochs repeat passes over the dataset, while mini-batches split data into manageable pieces.
- Validation data helps us measure generalization and detect overfitting.

Why scaling inputs helps

- Neural networks are usually optimized with gradient descent or one of its variants.
- If input features have very different scales, optimization becomes slower and less stable.
- Standardization is therefore a common and important preprocessing step.

Initialization

- If all weights start with the same value, neurons in the same layer stay identical during training.
- Random initialization breaks symmetry and allows different neurons to learn different features.
- Good initialization improves stability and makes optimization more effective.

Introductory hyperparameters

- The main introductory choices are number of layers, neurons per layer, activation, learning rate, and batch size.
- For a first model, simple defaults often work well: 1–2 hidden layers, ReLU, Adam, and early stopping.
- Hyperparameter tuning matters, but beginners should first build intuition with small working models.

How many hidden layers?

- For many practical problems, one or two hidden layers already provide strong performance.
- More layers can help on complex tasks, but they also make training and tuning harder.
- For first contact, shallow MLPs are the best starting point for understanding the basics.

How many neurons per layer?

- There is no universal rule, but hidden layers with a similar width often work well in practice.
- Too few neurons may underfit because the model lacks representational capacity.
- Too many neurons can increase overfitting risk, although regularization can mitigate this.

Learning rate

- The learning rate controls the size of each optimization step in parameter space.
- If it is too small, training is slow; if it is too large, the loss may oscillate or diverge.
- In practice, the learning rate is often the single most important hyperparameter.

Batch size

- A mini-batch is the subset of training examples used for one gradient update.
- Larger batches can be computationally efficient, but smaller ones sometimes generalize better.
- For beginners, moderate batch sizes are usually a good compromise between speed and stability.

Overfitting and validation

- Neural networks can fit training data very well, including noise, if they are too flexible.
- Validation curves help us see whether performance on unseen data is improving or degrading.
- Early stopping is a simple and powerful way to stop training before overfitting becomes severe.

A first mental picture of representation learning

- Lower layers often detect simple signals such as edges, trends, or local combinations of inputs.
- Intermediate layers combine these into more abstract patterns that are more useful for the task.
- The final layers convert these learned representations into predictions.

Practical beginner checklist

- Start with scaled data, a small MLP, ReLU hidden layers, and an output suited to the task.
- Use validation data, monitor loss and accuracy, and apply early stopping when possible.
- Only then try tuning depth, width, learning rate, batch size, and regularization.

Conclusions

- Perceptrons are linear; multilayer perceptrons become powerful thanks to hidden layers and nonlinear activations.
- Backpropagation plus gradient-based optimization is the engine that trains neural networks.
- Keras gives a simple practical interface for building the first useful neural network models.